

Droneboy Manual

Droneboy is a music application for the Game Boy system. It makes drones and chords and can play sequences of chords. It is also MIDI compatible with an Arduinoboy. Droneboy is also an open-source project available here:

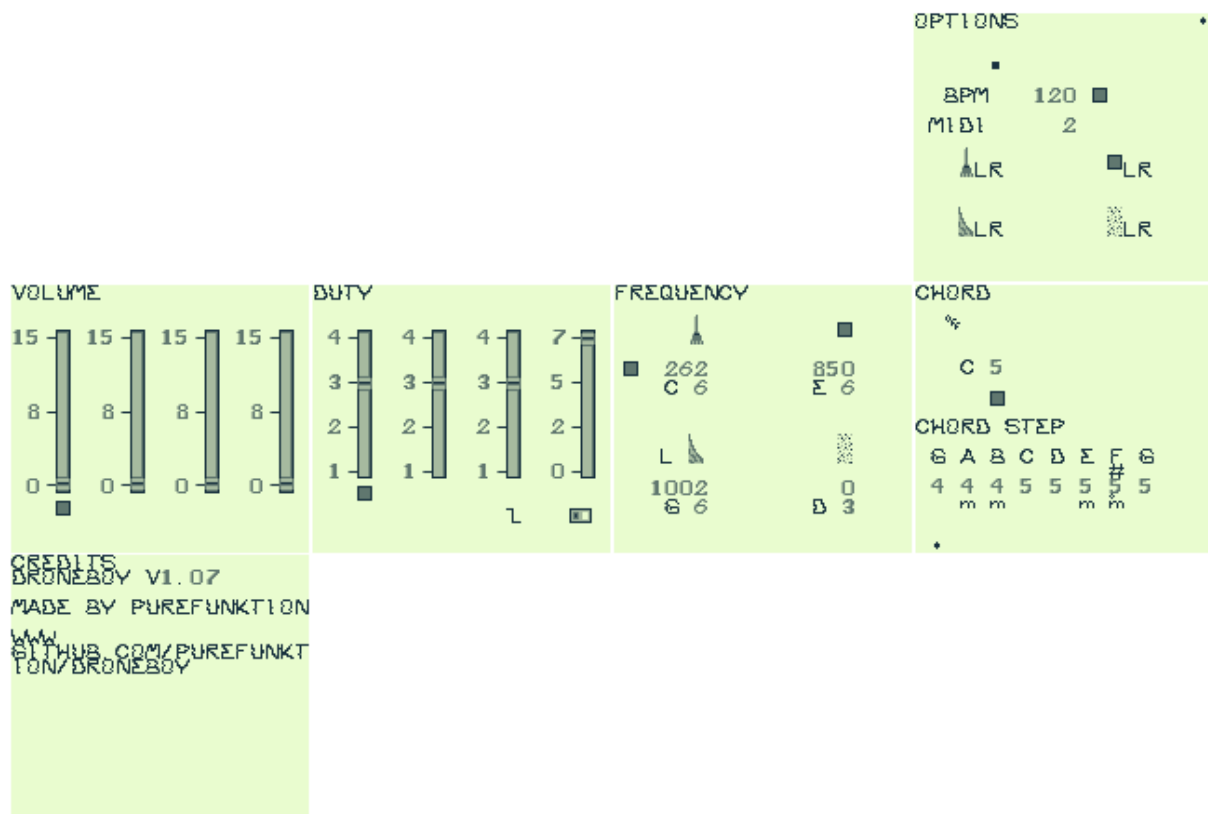
<https://github.com/purefunktion/Droneboy>

In Droneboy the 4 channels are called **Sweep**, **Square**, **Wave** and **Noise**. Usually the square channels are called “Pulse channel 1/2” or simply Pu1, Pu2. So Sweep = Pu1 and Square = Pu2.

Navigation

Droneboy consists of several control pages. To navigate between them hold **SELECT** and press any of **UP**, **DOWN**, **RIGHT** or **LEFT** on the D-Pad.

This is what the map looks like



You start out on the volume page and to get to the duty page you press **SELECT+RIGHT**. The map wraps around at volume and chord pages for easier navigation. From BPM page you can only navigate with **SELECT+DOWN** and so on.

Macro Markers

A common feature on most control pages is the macro and inverted macro markers. Use **A** button to place a macro marker under each fader/control. Press **A** again to place a inverted macro marker. And a third time to delete the marker.

When a macro marker is activated that fader will respond to whatever change is done on the other channels on that control page. For example a macro marker on the sweep channel on the volume page will increase in volume if the square (or any other) fader is increased.

An inverted macro marker will do the opposite, if another fader/control is increased the inverted one will decrease.

This feature allows you to control all channel volumes simultaneously or to adjust the octave of a minor chord on the frequency page, for example.

Volume page

RIGHT/LEFT change sound channel. The channels are from left to right sweep, square, wave and noise

UP/DOWN to increase/decrease volume, or **HOLD UP/DOWN** to continuously increase/decrease volume

HOLD B + UP/DOWN will maximize/minimize sound volume

Duty page

RIGHT/LEFT change sound channel.

Channels(left to right) sweep, square, wave, noise. The noise channel has no duty so the fader will instead control the "dividing ratio of frequencies" in the polynomial counter.

UP/DOWN to cycle through duty settings

B on noise channel will toggle the counter step of the polynomial counter.

A on wave channel will change wave type. There is a square, saw, ramp, triangle and sine wave to choose from. There is a high and low version of each wave type to choose from (done on frequency page). The high one mimics the square channels settings and the low ones are for lower frequencies. The high setting is also used when playing chords.

Frequency page

HOLD A and press B this changes the mode between notes and frequency

In frequency mode

RIGHT/LEFT change channel.

Top of the page sweep and square, and on the bottom wave and noise

UP/DOWN changes the frequency 1 step or **HOLD UP/DOWN** to continuously increase/decrease frequency

HOLD A + UP/DOWN changes frequency 100 steps

HOLD B + UP/DOWN changes frequency 10 steps

In note mode

RIGHT/LEFT change channel

UP/DOWN changes note 1 semitone

HOLD A + UP/DOWN changes note 12 semitones(octave)

HOLD B + UP/DOWN changes note 3 semitones

HOLD B and press A copies the frequency of the current note to the frequency section. An easy way to manipulate detuning etc.

The noise channel frequency mode controls the Shift Clock Frequency. The note mode has two notes that are "close enough" to authors ears of a D and G#.

PRESS A to toggle between high and low waves of the currently selected wave type (done on duty page)

Chord page

On the chord step page there are two different modes. One is the "chord change" mode and the other is for playing the chord step. When in chord change mode the marker(square) will be under the chord that is being manipulated. There are three parts of the chord that can be changed root note, minor/major and augmented/diminished/normal. Press **RIGHT/LEFT** to change between parts and up and down to manipulate the values. When on the root note part **HOLD A + UP/DOWN** to jump 12 semitones(octave) and **HOLD B + UP/DOWN** to jump 3 semitones. Just **UP/DOWN** will change the root note one semitone. To play the current chord press **A**, pressing **A** again won't turn the sound off but you can now change to a chord you like and then retrigger that by pressing **A** to "on" again. This is indicated by the on/off sprite above the chord change part.

The chord step part has 8 chord slots that can be filled with the chords you set in the chord change part. To make things a bit smoother there is a "record marker" that can be moved when in the chord change mode. By **HOLD A + LEFT/RIGHT** you can change the record marker's place in the sequence of chord slots. By pressing **B** you set the currently selected slot to the chord chosen in the chord mode. This way you can create a sequence of 8 chords.

To play the sequence you have to be in the chord play mode. Change mode by **HOLD A and press B** and the square marker will jump down to the chord step sequence. In chord step mode go **RIGHT/LEFT** to change slot and press **B** to play the current chord. The chord retriggers if you press **B** repeatedly on the same chord slot.

The chord sequencer plays the steps in sequence from left to right, by pressing **START** you can start the sequencer from anywhere in the program. The chord part has to be turned on(pressing **A** to set to on) to hear it. On the BPM page you can see the indicator for when the sequencer is on or off.

The sequence is played one quarter note per chord slot, so the sequencer is currently somewhat limited.

Options page

By pressing **RIGHT/LEFT** you cycle through the options.

First on the options page you can change the BPM that controls the chord sequencer. **UP/DOWN**

changes the beats per minute one step up or down.

On MIDI you can change the midi channel that Droneboy listens to. **UP/DOWN** selects between midi channel 1-16.

Then comes the panning which is laid out like the other pages sweep, square, wave and noise. On each you can have stereo(LR), hard right(R) or hard left(L). Change the value by moving **UP/DOWN**.

There is also an indicator to see if the sequencer is turned on or off and blinking icons to see the speed of the BPM setting. Turn sequencer on or off by pressing **START**.

CREDITS page

The credit page displays the version of Droneboy and the url to the source code.

MIDI

Using an **Arduinoboy** with **Mode 5** (mGB full midi) you can communicate with Droneboy and control various parts of the program. At the moment MIDI is only accepted on **channel one**.

In the release and under the folder “resources” in the repo you can find the file “droneboy.nktrl2_data”, a preset used for the KORG nanoKONTROL2 midi controller to be compatible with the CC layout of Droneboy (which is hardcoded). You can load this file to your nanoKONTROL2 device using the “KORG Kontrol Editor” provided for free by Korg.

Note On

Droneboy accepts Note On messages in a very crude way. The 8 chord slots are mapped from 1-8 starting from C4 (middle C) continuing up the white keys to C5.

NOTE OFF is ignored — infinite sustain, baby!

CC

Below list explains the configured CC numbers, what CC values Droneboy accepts and what happens. As stated above this is mapped out in the KORG nanoKONTROL 2 file. If you have another controller the following is what you have to specify.

CC 1 Sweep volume (0-15)

CC 2 Square volume (0-15)

CC 3 Wave volume (0-15)

CC 4 Noise volume (0-15)

CC 5 Sweep duty (0-3)

CC 6 Square duty (0-3)

CC 7 Wave duty 0-3 (only applicable on square wave type)

CC 8 Noise clock divider (0-7)

CC 9 Sweep frequency coarse (0-127), done in steps of 16 “period value”

CC 10 Square frequency coarse (0-127), done in steps of 16 “period value”

CC 11 Wave frequency coarse (0-127), done in steps of 16 “period value”

CC 12 Noise clock shift (0-15)

CC 13 Sweep frequency fine (0-127)

This takes an offset from the value set by the “coarse” value set by in this case CC 9. Then from 64 and upwards it adds to the offset with one each step, and decreases it from 63 and downwards. This means that centering the knob equals the same “period value” as the coarse setting, turning it clockwise increases frequency from the center and turning it counter clockwise decreases it.

CC 14 Square frequency fine (0-127), same as CC13 but for Square channel

CC 15 Wave frequency fine (0-127), same as CC13 but for Wave channel

CC 16-18 not used

CC 19 Momentary button to change wave type square, saw, ramp, triangle, sine and then wraps to square again.

CC 20 Toggle button Noise LSFR width on = 15 bit and off = 7 bit

CC 21 Toggle button mute/unmute Sweep channel

CC 22 Toggle button mute/unmute Square channel

CC 23 Toggle button mute/unmute Wave channel

CC 24 Toggle button mute/unmute Noise channel

CC 27 Toggle button switch between high and low version of wave type

CC 29 Momentary button rewind chord sequencer to first step

CC 32 Toggle button start/stop chord sequencer

CC 33 Toggle button chord mode on/off. This has to be on to hear the sequencer. This has to be off to use the frequency knobs.

Droneboy also supports NRPN messages

CC 38 is data entry LSB

CC 39 is data entry MSB

CC 98 is NRPN LSB (parameter number)

CC 99 is NRPN MSB (parameter number)

The available parameters are 1, 2 and 3 and they map to sweep, square and wave frequency, respectively. Set any value between 0-2047

Made with

GBDK-2020 (version gbdk-4.4.0)

<https://github.com/gbdk-2020/gbdk-2020>

Emulators used in development:

BGB – Superb debugger

<https://bgb.bircd.org/>

Sameboy – Excellent sound support

<https://sameboy.github.io/>

Emulicious – Great serial debugging

<https://emulicious.net/>

Sprites and maps made with GBTD/GBMB

<http://www.devrs.com/gb/hmgd/intro.html>

Invaluable reference document for Gameboy

<https://gbdev.io/pandocs/>

Sound table reference for Gameboy sound frequency/note/Hz/MIDI

<http://www.devrs.com/gb/files/sndtab.html>

Arduinoboy by Timothy Lamb

<https://github.com/trash80/Arduinoboy>

Serial implementation sources:

<https://github.com/trash80/mGB> (original)

Tim Stirrat's very helpful version: <https://github.com/tstirrat/mGB>

Special thanks to gwEm who made the lower wave tables and showed how to implement them in his fork of Droneboy

<https://github.com/gwEm303/Droneboy>